COMPUTER VISION INTERNSHIP ASSESSMENT — TECHNICAL REPORT

Egg Segmentation and Tracking

Tasks covered: pretrained segmentation, classical segmentation experiments, quantitative evaluation, and video tracking. Prepared from the project README and experiment notes.

1. Project Summary

This repository documents an end-to-end computer-vision assessment built around egg segmentation and tracking. The work was split into four connected tasks:

- **Task 1 - Pretrained segmentation:** generate egg masks using a pretrained segmentation pipeline.
- **Task 2 - Classical segmentation:** implement and compare image-processing approaches without using a trained egg detector.
- **Task 3 - Quantitative comparison:** evaluate candidate masks against reference masks and compare results with metrics and visual inspection.
- **Task 4 - Video tracking:** use the selected segmentation approach as the basis for detecting and tracking individual eggs in a video, with modifications where needed.

The project was approached experimentally rather than by choosing a model at the start. We inspected the data, generated reference/pseudo-reference masks, tried several segmentation methods, measured their behaviour, examined failure cases, then carried the most useful classical method into the video task.

> **Main takeaway:** Fixed-rectangle GrabCut was the best-performing tested classical segmentation method on the small still-image test split. For the video task, a lighter threshold-and-contour pipeline was explored because it was fast, interpretable, dependency-light, and could be tuned directly to the actual video. The video tracker is an approximate prototype, not a production-ready solution.

2. Why Classical Computer Vision Was Explored Instead of Relying on YOLO

The assessment was fundamentally about segmentation and tracking, not only object detection. A detector such as YOLO typically returns bounding boxes; it does not by itself provide the pixel-level egg masks needed for the segmentation tasks. A segmentation-capable model could provide masks, but would introduce model/runtime dependencies and would still need to be evaluated on the available images.

A pretrained YOLO11n detector was also tried separately. Bounding boxes were derived from pseudo-label masks and used to fine-tune the model. Its reported validation metrics were:

Precision	Recall	mAP@0.50	mAP@0.50:0.95
0.450	0.633	0.394	0.308

The resulting video annotations contained false detections and were not consistently satisfactory. This did not prove that YOLO is generally inferior; it showed that this particular fine-tuning setup, with a small dataset and pseudo-label-derived boxes, did not give sufficiently reliable results on this clip.

Classical methods were pursued for several practical and experimental reasons:

- **Task fit:** segmentation masks were needed for the first tasks; a bounding-box detector alone was not enough.
- **Small data:** the dataset contained only 110 images, and the reference masks were generated using a pretrained segmentation workflow rather than a large manually annotated set.
- **Transparency:** thresholding, contours, GrabCut, and shape filters make it possible to inspect why a region was selected or missed.
- **Low overhead:** the classical pipeline ran with OpenCV and did not need a detector inference loop for every video frame.
- **Direct tuning:** the eggs were bright against a dark tabletop in much of the clip, so a fixed threshold was a reasonable baseline to test.
- **Honest comparison:** the project retained the YOLO experiment as a comparison rather than claiming pretrained models were never explored.

Classical CV was not selected because it is universally better than deep learning. It was selected as the most practical route to an interpretable prototype for the conditions in this particular assessment, after the experiments showed both the strengths and weaknesses of the alternatives.

3. Data

Roboflow egg image dataset

- **Export metadata:** egg v1, dated 2025-05-23.
- **License shown in the export:** CC BY 4.0.
- **Total images:** 110.
- **Training split:** 75 images, plus an Unlabeled folder containing 2 images.
- **Validation split:** 22 images.
- **Test split:** 11 images.
- **Preprocessing:** auto-orientation and resize to 640×640 using stretch. No augmentation was applied.

The images contain white and brownish eggs, partially visible eggs, white backgrounds, and cases where egg boundaries are weak or blend into the background. This variation made it difficult to assume that one simple threshold would work for every image.

Video used for Task 4

- **Input video:** `data/Raw-and-Boiled-SnapYT.App.mp4`
- **Resolution:** 640×360. **Frame rate:** 24 FPS. **Duration:** approx. 60 seconds.
- **Scene:** two eggs on a dark tabletop, with a hand repeatedly moving into the scene and partially or fully covering the eggs.

The hand, reflections, bright clothing, and changes in egg position make video tracking harder than segmenting a clear still image.

4. Task 1 — Pretrained Segmentation and Reference Masks

A pretrained segmentation workflow involving DINO and SAM was explored to generate egg masks. Combined masks were stored under `outputs/egg_segmentation/combined`. Filenames followed a pattern similar to `train_egg_<image_stem>_combined.png`.

These masks were used as pseudo-reference masks for subsequent comparisons. They were useful for creating a consistent comparison target without manually drawing all masks, but they should not be treated as perfect ground truth: errors in the pretrained masks can affect the measured scores of the classical methods.

Why this task mattered

- It provided pixel-level masks to inspect and compare.
- It made it possible to evaluate classical segmentation methods against a common reference.
- It established a pretrained-model baseline for the assessment.

Important caveat: A metric measured against a pseudo-ground-truth mask measures agreement with that mask, not necessarily absolute biological correctness. Any missed edge or incorrect region in the reference mask can influence IoU and Dice.

5. Task 2 — Classical Segmentation Experiments

The purpose of this task was to test classical image-processing methods and understand which assumptions worked for the dataset. Several alternatives were tried rather than assuming one method would be best.

Methods explored

- **Otsu thresholding:** selects a threshold from the image intensity distribution. It was simple, but could include background highlights or fail when foreground and background intensities overlapped.
- **HSV-based segmentation:** explored colour-space thresholding as an alternative to grayscale intensity. Colour variation and background similarities limited its reliability.
- **K-means clustering:** explored grouping pixels by appearance. Clustering does not inherently know which cluster is the egg, so the relevant cluster still has to be identified and can vary between images.
- **Watershed:** explored separating regions using image boundaries/markers. It was sensitive to the quality of markers and could over-segment or fail on weak boundaries.
- **Canny edges:** explored edge-based extraction. Edges did not reliably form closed egg boundaries in these images.
- **GrabCut variants:** tested foreground/background segmentation using an initialization. We explored Fixed-rectangle GrabCut and Otsu-initialized GrabCut.
- **Morphological operations:** tested to clean masks, but closing sometimes merged nearby eggs or other foreground regions.

6. Task 3 — Quantitative Evaluation and Comparison

The task was not just to inspect masks visually; methods were compared quantitatively as well. The dataset had a 4:1 development/evaluation split requirement in the assessment workflow. The test split of 11 images was kept separate from tuning as far as the described workflow allowed.

The reported mask metrics were:

- **Intersection over Union (IoU):** overlap area divided by the union of the predicted and reference masks.
- **Dice coefficient:** twice the overlap divided by the total foreground pixels in both masks.

Both metrics range from 0 to 1, with higher values indicating greater overlap.

Method (11-Image Test Split)	IoU	Dice
Fixed-rectangle GrabCut	0.5397	0.6537
Otsu-initialized GrabCut	0.5014	0.5782
Canny edges	0.0483	0.0761

Results that guided the choice: Fixed-rectangle GrabCut was the strongest of these specifically reported comparisons. Canny performed very poorly because edge maps did not produce filled, accurate egg regions. Otsu-initialized GrabCut was weaker than fixed-rectangle GrabCut, suggesting that the Otsu initialization was not consistently giving GrabCut a useful foreground/background separation.

The fixed rectangle is an initialization prior: it assumes the object of interest lies inside a known region. It is therefore not a completely automatic, location-independent segmenter. The score is useful for this dataset and setup, but does not imply that it will generalize to arbitrary images. The quantitative results and qualitative inspection were used together; a method could produce a visually plausible result on one image but fail across the split.

7. Task 4 — Individual Egg Tracking in Video

Task 4 required tracking individual eggs in the provided video using the selected segmentation approach, with modifications permitted. The image segmentation approach could not be used unchanged: a video adds movement, temporary occlusion, and the need to preserve identities across frames.

Initial video observations

A representative frame around 20 seconds showed eggs against a dark tabletop. The white eggs were often brighter than the table, suggesting intensity thresholding was worth trying. However, a hand could cover an egg, and reflections could produce foreground regions.

Video pipeline experiments

1. Region of interest (ROI): The pipeline focused on the part of the frame where the tabletop and eggs generally appeared:

```
y1 = int(height * 0.38)
y2 = int(height * 0.82)
x1 = 0
x2 = int(width * 0.78)
```

This reduced irrelevant parts of the scene. It is a video-specific heuristic rather than a general object-localization method.

2. Grayscale and Gaussian blur: The blur reduced small intensity fluctuations before thresholding.

```
gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
blurred_roi = cv2.GaussianBlur(gray, (7, 7), 0)
```

3. Fixed binary threshold: A threshold of 155 often separated the white eggs from the darker table in clear frames. It was chosen empirically after inspecting the video rather than learned from training data.

```
mask_original = cv2.threshold(blurred_roi, 155, 255, cv2.THRESH_BINARY)[1]
```

Otsu thresholding and morphology were also inspected on video frames. Otsu sometimes picked up the hand and bright background regions, while closing could merge nearby egg regions. The fixed threshold gave a simpler and more useful baseline on the selected frames.

4. External contours: The contours were used to create candidate detections. Because every bright connected region is not an egg, contour filtering was necessary.

```
contours, _ = cv2.findContours(mask_original, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
```

5. Candidate selection: The first version removed contours below 100 pixels, sorted the remaining contours by area, and retained the two largest. This worked reasonably well in early portions of the clip, but could detect part of the hand or other bright regions. Shape filtering was then explored using contour area, bounding-box aspect ratio, and extent. A later per-frame experiment used a minimum area of 1500 pixels, aspect ratio between 0.65 and 1.6, and extent above 0.5. It reduced tiny fragments but could also reject the partly covered egg. These later filters were evaluated during debugging and should not be assumed to be in the saved full-video version unless the video loop was updated.

6. Identity assignment (left-to-right versus centroid tracking): The first identity strategy assigned Egg 1 and Egg 2 according to horizontal position. This was easy to implement but swapped IDs when the eggs changed horizontal order. A centroid-based association strategy was introduced instead:

- Compute the centre of each detected bounding box.
- Compare candidate centres with the previous centres of the two tracks.
- When two detections exist, compare both possible assignments and choose the one with the lower total movement.
- When only one detection exists, associate it with the nearest previous track if within a maximum distance (`MAX_DISTANCE = 70` pixels).
- Retain a missing track temporarily instead of dropping it immediately (up to `MAX_MISSING = 12` frames).
- Keep exactly two track IDs.

This reduced some ID swapping, but it cannot fully resolve long occlusions or incorrect detections.

7. Inspecting the failure after approximately 28 seconds: The output was reported to look good until around 28 seconds, after which annotations became irregular. Frames at 27, 29, and 31 seconds were inspected. At 29 seconds, one egg remained clearly visible while the other was largely covered by the hand. With threshold 155, the visible egg formed a large, coherent white contour, but the covered egg was split into small white fragments around the fingers. This explains why a contour tracker can suddenly select fragments or lose the partly hidden egg: the segmentation mask no longer contains a clean egg-shaped region.

A shape filter was tested. It retained the visible egg and a small fragment, demonstrating that geometry filters can remove some false positives but cannot reconstruct an egg that is mostly hidden. The sensible behaviour during full occlusion is to keep the track identity temporarily without drawing a confident box around a fragment.

8. Output: The described full-video output was saved as `outputs/egg_tracking_classical.avi`. The AVI was written using MJPG. FFmpeg was not installed, and the workflow avoided installing additional libraries just for video conversion.

8. Summary of Key Decisions

Decision	Reason
Use DINO/SAM for reference masks	Needed pixel masks for comparison without manually annotating every image.
Compare multiple classical methods	No single method was assumed to work; image boundaries and backgrounds varied.
Measure IoU and Dice	Needed quantitative comparison in addition to visual inspection.
Prefer fixed-rectangle GrabCut for stills	Best reported IoU/Dice among the specifically compared still-image methods.
Try YOLO11n separately	Establish a pretrained detector comparison for the video/object-detection side.
Continue with classical CV for video	YOLO output had false detections; thresholding worked on clear frames and was easier to tune.
Use a fixed threshold of 155 for video	Empirically separated eggs from the tabletop in inspected frames.
Use an ROI	Reduced irrelevant areas of the frame.
Replace left-to-right IDs with centroid matching	Left-to-right ordering caused ID swaps when eggs crossed or changed order.
Retain missing tracks briefly	Short occlusions should not immediately destroy an egg's identity.

Decision	Reason
Avoid claiming perfect tracking	The hand makes segmentation ambiguous and the output degrades after about 28 seconds.

9. Limitations and Honest Results

- Still-image segmentation scores are based on a small test split and pseudo-reference masks.
- Fixed-rectangle GrabCut uses a spatial initialization and is not fully automatic.
- The classical video pipeline is based on brightness and contours, not a learned egg classifier.
- Hand occlusion breaks the egg mask into fragments or hides it entirely. The tracker can still lose or swap IDs when detections are ambiguous.
- There is no formal video benchmark or verified frame-by-frame tracking score. The YOLO validation metrics are separate from the classical video result.
- The geometric filter was tested on difficult frames; confirm the final notebook before assuming it was used for the exported video.

10. Reproducibility Settings

Parameter	Value	Parameter	Value
Video resolution	640 × 360	**Gaussian blur**	7 × 7
Video FPS	24	**Fixed threshold**	155
Approximate duration	60 seconds	**Initial minimum contour area**	100 pixels
ROI vertical range	38% – 82% of height	**Number of track IDs**	2
ROI horizontal range	0% – 78% of width	**Centroid association distance**	70 pixels
Output	`outputs/egg_tracking_classical.avi`	**Maximum missed frames retained**	12

These values summarize the latest described pipeline. Some experiments were performed in separate notebook cells, so inspect the final script/notebook to confirm which settings were active when the output video was generated.

11. Potential Next Steps

Not implemented, but possible future improvements:

- Improve illumination handling or use adaptive segmentation.
- Use local search around the last known egg position during occlusion.
- Use expected egg size and shape from clear frames.
- Combine appearance and motion information.
- Manually label a small video subset to measure detection and ID-switch performance.
- Compare with a detector trained on manually verified bounding boxes.
- Evaluate a segmentation-capable model if pixel-level video masks are required.

12. Technologies and Output Video

Technologies: Python, OpenCV, NumPy, Matplotlib, Jupyter Notebook, DINO/SAM for mask generation experiments, and Ultralytics YOLO11n for the separate detector experiment.

Output video: https://drive.google.com/file/d/1vgV520dUHwrknSW97UzGkZe3Cm_4F9KH/view?usp=sharing

13. AI-Assisted Development and Human Decision-Making

AI tools were used throughout this project to support coding, research, experimentation, and report preparation. ChatGPT was my primary tool, and I also used Gemini and Claude to explore different perspectives, discover alternative methods, and make my research more diverse.

How I used AI

- **Code assistance:** helped write, understand, debug, and improve code for computer vision experiments.
- **Research:** explored different segmentation, detection, and tracking methods to identify approaches worth testing.
- **Experimentation:** discussed possible improvements, parameter choices, and reasons behind unexpected results.
- **Report preparation:** used AI to restructure the report, improve readability, organize findings, and communicate technical decisions more clearly.

My contribution and decision-making

Although AI supported the workflow, I did not rely on its suggestions blindly. My main contribution was making technical decisions, running experiments, inspecting outputs, and evaluating results using metrics such as IoU and Dice. I decided which approaches were worth exploring, when an experiment had a realistic chance of improvement, and when it was more useful to stop an approach and move on to another. I also examined failure cases to understand why a method was not working as expected and used those observations to guide the next experiment.

AI-generated code and recommendations were treated as suggestions that needed to be tested and verified. The final decisions were based on actual experiment results, visual inspection, quantitative evaluation, and practical trade-offs rather than AI recommendations alone. This approach allowed me to use AI as a productivity and research assistant while keeping the experimentation, evaluation, critical thinking, and decision-making central to my own work.

14. Conclusion

The assessment was completed as an iterative investigation rather than a single-model implementation. We first explored pretrained segmentation masks, then compared classical segmentation methods quantitatively, and finally adapted a simple classical pipeline to a moving video.

Fixed-rectangle GrabCut had the strongest reported still-image metrics among the named comparisons, while thresholding and contour tracking were chosen for the lightweight video prototype because they worked on clear frames and were easy to inspect.

The project demonstrates method comparison, metric-based evaluation, debugging through visual failure analysis, and a reasoned trade-off between pretrained models and classical computer vision. The remaining weakness is occlusion: when a hand covers an egg, the pixels needed for a reliable segmentation are not visible, and simple contours cannot infer the missing shape.